

Praktikum Pemrograman Berbasis Objek

Pertemuan 3

Materi Pertemuan 3

1. Inheritance

2. this & super keywords

3. Overriding

4. Overloading

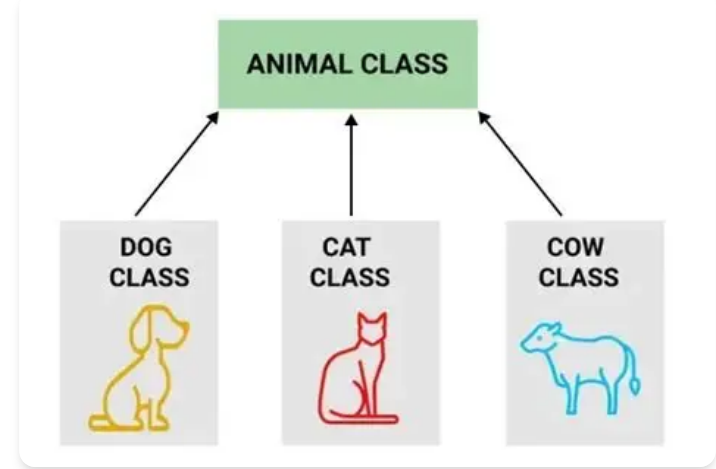
Inheritance

Mewarisi karakteristik dari sebuah class

Definisi Inheritance

Secara bahasa, inheritance adalah pewarisan. Inheritance adalah mekanisme untuk mewarisi karakteristik dari sebuah class. Dari sisi penulis kode, inheritance dapat digunakan untuk mengembangkan fitur di class baru (child class) dari class asalnya (parent class). Dan biasanya (meski tidak selalu), terdapat hubungan yang semantik antara parent dan child class.

Merupakan salah satu konsep di OOP yang menyatakan bahwa suatu class dapat memiliki class turunan (sub-class). Secara umum, konsep inheritance ini digunakan untuk menerima properti atau karakteristik dari class lain yang memiliki hubungan is-a.



Mengapa konsep inheritance muncul?

Konsep inheritance pertama kali muncul pada tahun 1960-an melalui bahasa pemrograman Simula 67, yang dirancang untuk memodelkan fenomena objek di dunia nyata. Di tengah kompleksitas pemodelan tersebut, inheritance hadir sebagai solusi untuk mereduksi duplikasi kode dan memfasilitasi klasifikasi taksonomi.

Namun, hal ini memunculkan miskonsepsi: inheritance sering dianggap murni sebagai fasilitas desain di mana setiap objek wajib memiliki hubungan representatif (hubungan is-a), padahal hal tersebut tidak sepenuhnya benar.

Dalam Java, inheritance ditandai dengan penggunaan kata kunci `extends`. Kata kunci ini digunakan untuk menyatakan bahwa suatu class merupakan turunan dari class lain.

```
public class Mobil extends Kendaraan {  
  
}
```

Pada contoh di atas, `extends` dipakai untuk menunjukkan bahwa class `Mobil` mewarisi seluruh karakteristik (atribut dan method) dari class `Kendaraan`.

Kapan Inheritance digunakan?

1. Kebutuhan untuk modifikasi incremental, pada definisi dijelaskan jika konsep ini memungkinkan untuk mengembangkan fitur di class baru, adaptasi ini memungkinkan modifikasi dengan membuat definisi baru dan menjadi bagian sistem lama tidak rusak.
2. Terdapat struktur taksonomi (is-a relationship) pada sistem yang dikembangkan, kasus paling umum di mana inheritance digunakan ketika sebuah class memiliki hubungan representatif dengan class lain.
3. Kebutuhan untuk code reusability, meski tidak memiliki hubungan konseptual yang nyata, inheritance dapat digunakan untuk code reuseability jika sebuah class memiliki karakteristik yang mirip dengan class lain.

Tujuannya apa sih:

- DRY (Don't Repeat Yourself)
- Code Reusability

Dengan adanya konsep ini lebih memudahkan programmer jika menghadapi situasi dimana ada 2 class berbeda yang memiliki behaviour yang sama sehingga method tidak perlu ditulis ulang dengan catatan kedua class tersebut memiliki hubungan is-a.

Catatan mengenai inheritance

Java hanya dapat mewarisi dari satu class saja, berbeda dengan C++ yang memungkinkan melakukan pewarisan dari dua atau lebih class. Namun, multi inheritance menimbulkan masalah baru, yaitu diamond problem.

Batasan Inheritance:

- **Diamond Problem** -> permasalahan yang terjadi ketika child-class memiliki lebih dari 1 parent-class (multiple inheritance) dimana child-class meng-inherit properties dan methods dari 2 parent class yang memiliki method dengan nama dan parameter yang sama. Karena ada diamond problem ini, maka java tidak mengizinkan adanya Multiple Inheritance satu child-class di java hanya boleh memiliki 1 parent-class.
- **Harus memperhatikan access modifier** dengan teliti sebelum menurunkan suatu class. Sebagai contoh jika di parent-class memiliki atribut dengan access modifier private, maka atribut tersebut tidak dapat diakses secara langsung.

Contoh penerapan inheritance

(studi kasus is-a relationship / general to specialization)

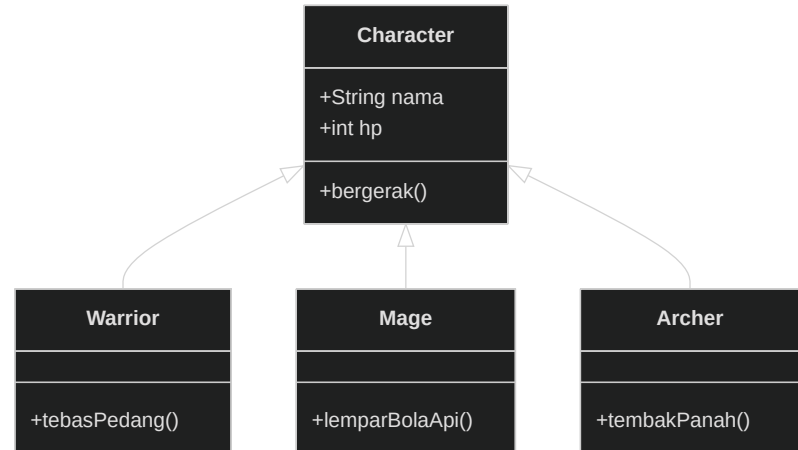
Misal diketahui sebuah sistem game yang terdapat class

`Character` sebagai sistem dasar (*parent class*) untuk sistem turunannya.

Game tersebut memiliki tiga spesialisasi (*child class*):

- Warrior (pengguna pedang)
- Mage (pengguna sihir)
- Archer (pengguna busur & serangan jarak jauh)

Setiap spesialisasi mewarisi atribut dasar dari `Character` (seperti HP atau nama), tapi memiliki karakteristik serangan yang berbeda-beda.



Implementasi: Parent Class (Character)

Pertama, kita definisikan class `Character` yang bertindak sebagai fondasi utama. Class ini mendefinisikan atribut dan *method* dasar yang secara umum pasti dimiliki oleh setiap karakter di dalam game.

```
// Parent Class
public class Character {
    public String nama;
    public int hp; // Health Point

    // Method umum yang bisa diwariskan
    public void bergerak() {
        System.out.println(nama + " bergerak maju.");
    }
}
```

Implementasi: Child Class (Warrior, Mage, Archer)

Dengan menggunakan keyword `extends`, class turunan secara otomatis mewarisi atribut dan `bergerak()` dari class `Character`, lalu menambahkan serangan spesifik mereka sendiri.

```
public class Warrior extends Character {  
    public void tebasPedang() {  
        System.out.println(nama + " menebas dengan pedang!");  
    }  
}
```

```
public class Mage extends Character {  
    public void lemparBolaApi() {  
        System.out.println(nama + " melempar bola api!");  
    }  
}
```

```
public class Archer extends Character {  
    public void tembakPanah() {  
        System.out.println(nama + " menembakkan panah!");  
    }  
}
```

Access modifier di inheritance

Access modifier tidak terbatas pada public dan private saja, **protected** kadangkala digunakan di inheritance, tujuannya agar class turunannya dapat mengakses langsung atribut dan/atau method dari class parentnya tanpa memerlukan getter dan setter.

Meski praktik ini, dianggap melanggar encapsulation.

Access Modifier	Class yang Sama	Package yang Sama	Subclass	Class Manapun
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

this & super keywords

Mereferensikan objek dari class itu sendiri dan parent class

Kata kunci this & super

- `this` Keywords -> tugasnya adalah menunjuk atribut (variable) atau method milik class itu sendiri
- `super` Keywords -> tugasnya adalah menunjuk atribut (variable) atau method milik parent-classnya

Ketika menulis kode di child class, seringkali kita menggunakan `super` . `super` adalah kata kunci yang merepresentasikan objek dari parent class.

`super` sama seperti `this` atau nama variabel yang didefinisikan sebagai objek dari sebuah class. `this` merepresentasikan objek di class itu sendiri, `super` merepresentasikan objek untuk parent classnya.

Contoh this & super (Studi Kasus Game)

Mari kita terapkan **constructor** pada studi kasus karakter game sebelumnya:

- `this.nama` dipakai di class `Character` untuk menegaskan bahwa kita merujuk pada atribut milik **class** tersebut (bukan parameter).
- `super(nama, hp)` di class `Warrior` berfungsi memanggil **constructor** milik **parent class** (`Character`) untuk menginisialisasi atribut dasar.
- `super.nama` memanggil atribut dari **parent** secara eksplisit (walaupun bisa dipanggil langsung tanpa ``super`` jika tipenya ``protected``).

```
class Character {
    protected String nama;
    protected int hp;

    public Character(String nama, int hp) {
        // 'this' membedakan atribut milik class dengan param
        this.nama = nama;
        this.hp = hp;
    }
}

class Warrior extends Character {
    private String tipePedang;

    public Warrior(String nama, int hp, String tipePedang) {
        // 'super' memanggil dan mengirim data ke constructor
        super(nama, hp);
        this.tipePedang = tipePedang;
    }

    public void info() {
        System.out.println("Nama: " + super.nama);
        System.out.println("Senjata: " + this.tipePedang);
    }
}
```

Method Overriding

Mendefinisikan ulang implementasi dari method

Method Overriding

Definisi

Method overriding lebih kayak nulis ulang method yang udah ada sehingga memungkinkan subclass untuk menyediakan implementasi spesifik dari sebuah metode yang sudah didefinisikan di kelas induknya (parent class).

Pada contoh sebelumnya, dikatakan bahwa setiap karakter memiliki karakteristik serangan yang berbeda. Karakteristik yang berbeda dari class utama ini dapat diimplementasikan dengan konsep method overriding.

Mekanismenya adalah mendefinisikan ulang method dengan nama, return type, dan parameter yang sama, tetapi penerapan di dalamnya berbeda. **Wajib tambahkan** `@Override` agar Java tahu ada method yang ditimpa.

Access modifier

Access modifier di sub-classnya tidak boleh lebih ketat dari parent class.

Contoh Method Overriding (Studi Kasus Game)

Mari kita terapkan pada class `Character` . Kita membuat method `serang()` di *parent class*, lalu melakukan *override* di *child class* (misalnya `Warrior`) agar serangannya spesifik.

Penjelasan & Output:

Saat kita memanggil method `serang()` dari objek `Warrior` , Java mengeksekusi implementasi milik `Warrior` (karena sudah ditimpa). Output yang dihasilkan:

```
> Warrior menebas dengan pedang besar!
```

```
class Character {  
    // Method dasar yang akan dioverride  
    public void menyerang() {  
        System.out.println("Karakter memukul biasa ...");  
    }  
}  
  
class Warrior extends Character {  
    // Menimpa method menyerang() milik Character  
    @Override  
    public void menyerang() {  
        System.out.println("Warrior menebas dengan pedang besar!");  
    }  
}
```

Method Overloading

Satu nama method dengan variasi parameter

Method Overloading

Definisi

Sebuah konsep dimana ada beberapa method dengan nama yang identik tetapi menerima input atau parameter yang berbeda.

Method overloading adalah mekanisme di mana kita bisa menulis ulang method dengan nama yang sama, tetapi dengan return type, tipe data parameter, urutan parameter, atau jumlah parameter yang berbeda serta implementasi method yang berbeda juga.

Method Overloading (Lanjutan)

- Salah satu fungsionalitas dari Java dimana satu class bisa memiliki beberapa method (fungsi) dengan nama yang sama tetapi parameter yang berbeda-beda.
- Compiler akan secara otomatis menentukan fungsi mana yang paling cocok dengan inputan yang diberikan.

Syarat:

- List parameter harus **berbeda** (jumlah, tipe data, urutan).
- **TIDAK BOLEH** overload hanya berdasarkan perbedaan return type (untuk overloading method).

Implementasi method overloading pada attack()

Karakter pada sebuah game mungkin memiliki kemampuan serangan yang berbeda-beda. Jika kita ingin memperluas kemampuannya, kita punya dua opsi: menambah method baru yang berbeda, atau mengimplementasikan *method overloading*.

Misal, sebuah karakter memiliki mekanisme serangan yang berbeda tergantung aksi yang dikirim pengguna. Alih-alih menulis method baru yang membingungkan, kita dapat memakai nama method yang sama (`attack`). Jika ada kondisi tertentu, alurnya juga bisa dialihkan ke method lain yang tujuannya sama-sama untuk menyerang.

```
class Character {  
  
    // 1. Serangan dasar (tanpa parameter)  
    public void attack() {  
        System.out.println("Karakter memukul dengan tangan ko  
    }  
  
    // 2. Overload: Serangan dengan senjata  
    public void attack(String senjata) {  
        System.out.println("Karakter menyerang dengan " + sen  
    }  
  
    // 3. Overload: Serangan dengan senjata dan sihir  
    public void attack(String senjata, String elemen) {  
        // Pemanggilan method attack(String) di dalam overloa  
        this.attack(senjata);  
        System.out.println("→ Memberikan damage tambahan ber  
    }  
}
```

Overloading (constructor)

- *Overloading* pada *constructor* berarti membuat banyak konstruktor yang memiliki jumlah atau tipe parameter yang berbeda-beda.
- Pada studi kasus game kita, beberapa karakter (seperti NPC) mungkin hanya butuh diinisialisasi menggunakan standar bawaan, sementara karakter *boss* butuh ditentukan status HP-nya secara spesifik di awal.

```
public class Character {  
    protected String nama;  
    protected int hp;  
  
    // 1. Default (tanpa parameter)  
    public Character() {  
        this.nama = "NPC Misterius";  
        this.hp = 100;  
    }  
  
    // 2. Hanya parameter nama (HP otomatis)  
    public Character(String nama) {  
        this.nama = nama;  
        this.hp = 100; // nilai default  
    }  
  
    // 3. Parameter nama dan hp (custom lengkap)  
    public Character(String nama, int hp) {  
        this.nama = nama;  
        this.hp = hp;  
    }  
}
```

Latihan Minggu 3

note : edit ini

Assignments

note : edit ini

Teknis Pengumpulan

Teknis pengumpulan tugas akan diinformasikan dikemudian hari.

Deadline Pengumpulan

Kelas A:

07 September 2026

Kelas B:

06 September 2026

Kelas C:

08 September 2026

Waktu yang dilihat adalah waktu last commit. Jika ada yang commit melewati deadline walaupun sudah commit sebelumnya akan dianggap terlambat.

Terima Kasih!

Do you have any questions? Please use respective class discussion channel on Discord.

Semangat terus menjalani kuilahnya! 🔥🔥🔥